Segment 4: Operator Overloading

Contents

- Operator Overloading
- Binary operator overloading
- Unary operator overloading
- Relational and logical operator overloading
- Operator overloading using friend functions
- Limitations of operator overloading

Operator Overloading
Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is an idea of giving special meaning to an existing operator in C++ without changing its original meaning. The mechanism of giving special meaning to an operator is known as operator overloading.

For example,
```
int a;
float b, sum;
sum=a+b;
```
Here, variables "a" and "b" are of types "int" and "float", which are built-in data types. Hence the addition operator '+' can easily add the contents of "a" and "b". This is because the addition operator "+" is predefined to add variables of built-in data type only.
Now, consider another example
```
class A
{
//Body of class A
};
int main()
{
A a1,a2,a3;
a3= a1 + a2;
return 0;
}
```
In this example, we have 3 variables "a1", "a2" and "a3" of type "class A". Here we are trying to add two objects "a1" and "a2", which are of user-defined type i.e. of type "class A" using the "+" operator. This is not allowed, because the addition operator "+" is predefined to operate only on built-in data types. But here, "class A" is a user-defined type, so the compiler generates an error. This is where the concept of "Operator overloading" comes in. Now, if the user wants to make the operator "+" to add two class objects, the user has to redefine the meaning of the "+" operator such that it adds two class objects. This is done by using the concept "Operator overloading". Redefining the meaning of operators really does not change their original meaning; instead, they have been given additional meaning along with their existing ones.

Sytax of operator overloading

1. Syntax for Operator Overloading using member function

ReturnType **operator** symbol (className const &arguments) {
//Statements;
}

2. Syntax for Operator Overloading using Friend function

friend return-type **operator** symbol (Variable 1, Varibale2)
{
//Statements;
}

Sample Question:

1. Write a syntax of overloading += operator using member function and using friend function [Aut-23]
2. Define Operator Overloading.Difference between Member function and Friend Function.[Sp-23]
3. Define operator overloading? Write the rules of operator overloading [Sp-22]

Operators that can be overloaded
• Binary Arithmetic -> +, -, *, /, %
• Unary Arithmetic -> +, -, ++, —
• Assignment -> =, +=,*=, /=,-=, %=
• Dynamic memory allocation and De-allocation -> New, delete
• Subscript -> [ ]
• Function call -> ()
• Logical -> &, | |, !
• Relational -> >, < , = =, <=, >=
Operator that cannot be overloaded are as follows:
• Scope operator (::)
• Sizeof
• member selector(.)
• member pointer selector(*)
• ternary operator(?:)

Operator Overloading in Unary Operators
The unary operators operate on a single operand.When you overload a unary operator using a member function, the function has no parameters. Since there is only one operand, it is this operand that generates the call to the operator function. There is no need for another parameter.

Following are the examples of Unary operators
➢The increment (++) and decrement (--) operators.
➢The unary minus (-) operator.
➢The logical not (!) operator.

C++ Program to overload ++ when used as prefix such as ++X

```
#include <iostream>
using namespace std;
class Count {
private:
int value;
public:
Count () {
value=5;
}
// Overload ++ when used as prefix
Count operator ++ () {
++value;
}
void display() {
cout << "Count: " << value << endl;
}
};
int main() {
Count count1;
// Call the "void operator ++ ()" function
++count1;
count1.display();
return 0;
}
```

Count() : value(5) {}

The above example works only when ++ is used as a prefix. To make ++ work as a postfix we use this syntax.

```
Count operator ++ (int) {
// code
}
```

Here, the int inside the parentheses. It's the syntax used for using unary operators as postfix; it's not a function parameter.

Example
Overload ++ when used as prefix and postfix

```
#include <iostream>
using namespace std;

class Count {
private:
int value;
public:
Count() {
Value=5;
}
//Overload ++ when used as prefix
Count operator ++ () {
```

```
++value;
}
// Overload ++ when used as postfix
Count operator ++ (int) {
value++;
}
void display() {
cout << "Count: " << value << endl;
}
};

int main() {
Count count1;
// Call the "void operator ++ (int)" function
count1++;
count1.display();
// Call the "void operator ++ ()" function
++count1;
count1.display();
return 0;
}
```

Sample Question:

1. Overload -(negative operator for both as binary operator and as a unary operator.[Sp-23]
2. Write a program to overload ++ using member function[Sp-23]
3. Suppose you have a class named Temperature that represents Temperature values.Implement the Temperature class and overload the decrement -- operator using a friend function [Aut-23]

Operator Overloading in Binary Operators

When a member function overloads a binary operator, the function will have only one parameter. This parameter will receive the object that is on the right side of the operator. The object on the left side is the object that generates the call to the operator function and is passed implicitly by **this.**

(+) Operator Overloading

C++ Program to Demonstrate Binary Operator Overloading

```
#include <iostream>
using namespace std;

class Complex {
private:
   int real=10, image=5;

public:
```

```
   Complex(int r, int i)
   {
      real = r;
      image = i;
   }

  Complex operator +(Complex const& obj)
   {
      Complex res;
      res.real = real + obj.real;
      res.imag = imag + obj.imag;
      return res;
   }
   void print() {

            cout << real << " + i" << image << '\n'; }
};

int main()
{
   Complex c1(10, 5), c2(2, 4);
   Complex c3 = c1 + c2;
   c3.print();
}
```

C++ Program to Demonstrate Binary Operator Overloading

```
#include<iostream>
using namespace std;
class Weight
{
 private:

 int kilo;

public:
Weight()
{
 kilo = 5;
}
Weight(int k)
{
kilo = k;
}

Weight operator + (Weight const  &obj)
{
int total;
total = kilo + obj.kilo;
cout<<"Total: "<<total<<endl;
}
```

```
};

 int main()
{
Weight w1,w2(20);
Weight w3;
w3 = w1 + w2;
return 0;
}
```

Remember: When a binary operator is overloaded, the left operand is passed implicitly by **this** pointer to the function and the right operand is passed as an argument.

Assignment Operator (=) Overloading

```
#include<iostream>
using namespace std;
class Distance {
private:
   int feet;
   int inches;
public:
Distance() {
feet = 0;
inches = 0;
}
Distance(int f, int i) {
feet = f;
inches = i;
}
Distance operator = (Distance const  &D ) {
feet = D.feet;
inches = D.inches;
}
// method to display distance
Distance  displayDistance() {
cout << "F: " << feet
cout << " I:" << inches << endl;
}
};
int main() {
   Distance D1(11, 10), D2(509, 119);
   cout << "First Distance : ";
   D1.displayDistance();
   cout << "Second Distance :";
   D2.displayDistance();
   // use assignment operator
   D1 = D2;
   cout << "First Distance :";
   D1.displayDistance();
return 0;
```

```
}
```

(/) Operator Overloading

```
#include<iostream>
using namespace std;
class Weight
{ private: int kilo;
public:
Weight()
{
kilo = 25;
}
Weight(int k)
{
kilo = k;
}
Weight operator / (Weight  const &obj)
{
int total;
total = kilo / obj.kilo;
cout<<"Total: "<<total<<endl;
}
};
int main()
{
Weight w1,w2(5);
Weight w3;
w3 = w1 / w2;
return 0;
}
```

Sample Question:

1. Overload -(negative operator for both as binary operator and as a unary operator.[Sp-23]
2. Write a program of using the following statement with a c++ program
   Ob2=50+ob1.[Sp-23]
3. Write a c++ program that showcase the usage of the operator overloading function for the complexNumber class.Add two ComplexNumber objects using overloaded + operator[Aut-23]

Relational and logical operator overloading

- There are various relational operators supported by C++ language like (<, >, <=, >=, ==, etc.) which can be used to compare C++ built-in data types. You can overload any of these operators, which can be used to compare the objects of a class.
- When you overload the relational and logical operators so that they behave in their traditional manner, you will not want the operator functions to return an object of the class for which they are defined. Instead, they will return an integer that indicates either true or false.
- An overloaded relational or logical operator function return a value of type **bool**, the **bool** type defines only two values: **true** and **false**. These values are automatically converted into nonzero and 0 values. Integer nonzero and 0 values are automatically converted into **true** and **false**.

Following example explains how a < operator can be overloaded and similar way you can overload other relational operators

```
#include <iostream>
using namespace std;
class Distance {
  private:
    int feet;
    int inches;
  public:
    Distance(int f, int i) {
      feet = f;
      inches = i;
    }
    // method to display distance
    void displayDistance() {
      cout << "F: " << feet << " I:" << inches <<endl;
    }
    // overloaded < operator
    bool operator <(const Distance& d) {
      if(feet < d.feet) {
        return true;
      }
      if(feet == d.feet && inches < d.inches) {
```

```
        return true;
      }

      return false;
    }
};

int main() {
  Distance D1(11, 10), D2(5, 11);

  if( D1 < D2 ) {
    cout << "D1 is less than D2 " << endl;
  } else {
    cout << "D2 is less than D1 " << endl;
  }
  return 0;
}
```

Output:
D2 is less than D1

Sample Question:

1.Construct a program to overload logical AND (&&) operator using member function.

2. Write a c++ program that showcase the usage of the operator overloading functions for the Vector class.Perform operations such as inequality between two vector objects using the overloaded != operator.

Operator Overloading in C++ Using Friend Function

Points to Remember While Overloading Operator using Friend Function:

1. The Friend function in C++ using operator overloading offers better flexibility to the class.
2. The Friend functions are not a member of the class and hence they do not have 'this' pointer.
3. When we overload a unary operator, we need to pass one argument.
4. When we overload a binary operator, we need to pass two arguments.
5. The friend function in C++ can access the private data members of a class directly.
6. An overloaded operator friend could be declared in either the private or public section of a class.

7. You cannot use a friend to overload the assignment operator. The assignment operator can be overloaded only by a member operator function.

Syntax to use Friend Function in C++ to Overload Operators:

```
friend return-type operator operator-symbol (Variable 1, Varibale2)
{
//Statements;
}
```

Example:

```
#include <iostream>
using namespace std;
class Complex
{
   private:
      int real;
      int img;
   public:
      Complex (int r = 0, int i = 0)
      {
         real = r;
         img = i;
      }
      void Display ()
      {
         cout << real << "+i" << img;
      }
      friend Complex operator + (Complex c1, Complex c2);
};

Complex operator + (Complex c1, Complex c2)
{
   Complex temp;
   temp.real = c1.real + c2.real;
   temp.img = c1.img + c2.img;
   return temp;
}

int main ()
{
   Complex C1(5, 3), C2(10, 5), C3;
   C1.Display();
   cout << " + ";
   C2.Display();
   cout << " = ";
   C3 = C1 + C2;
   C3.Display();
}
```

Limitations of Operator Overloading

1. Only built-in operators can be overloaded. New operators can not be created
2. Arity of the operators cannot be changed.
3. Overloaded operators cannot have default arguments except the function call operator () which can have default arguments.
4. The overloaded operator contains atleast one operand of the user-defined data type.
5. We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.

Sample Question:

1.Suppose you have a class named Temperature that represents Temperature values.Implement the Temperature class and overload the decrement -- operator using a friend function [Aut-23]

Previous Question Answer:

a) **Define operator overloading? Write the rules of operator overloading.**
Operator overloading is a feature in programming languages that allows the same operator to have different behaviors depending on the operands used.

The rules of operator overloading includes:

Operator Availability: Not all operators can be overloaded in every programming language. Each language defines which operators can be overloaded and the specific rules for each.

Number of Operands: Most unary and binary operators can be overloaded. Unary operators like increment (++), decrement (--), and logical negation (!) can be overloaded with only one operand. Binary operators like addition (+), subtraction (-), and equality (==) can be overloaded with two operands.

Operand Types: Operator overloading can be performed with user-defined types (classes) or built-in types (integers, strings, etc.). However, some languages have restrictions on which operators can be overloaded for built-in types.

b) Create a class float that contains one float data member. Overload an arithmetic operator so that it can operate on the objects of float.

```
#include<iostream.h>
#include<stdio.h>
#include<conio.h>
class Check
{
private:
float x;
```

```
public:
void setData(int a)
{
x=a;
}
void getData()
{
cout<<"\nx:"<<x;
}

//Defining an operator function + to overload
Check operator +(Check c)
{
Check temp;
temp.x=x+c.x;
return temp;
}

//Defining an operator function - to overload
Check operator -(Check c)
{
Check temp;
temp.x=x-c.x;
return temp;
}
//Defining an operator function * to overload
Check operator *(Check c)
{
Check temp;
temp.x=x*c.x;
return temp;
}

//Defining an operator function / to overload
Check operator /(Check c)
{
Check temp;
temp.x=x/c.x;
return temp;
}
};

void main()
{
clrscr();
Check c1,c2,c3,c4,c5,c6;
c1.setData(20);
c2.setData(10);
```

```
//Overloading operator +
c3=c1+c2;
c3.getData();

//Overloading operator -
c4=c1-c2;
c4.getData();

//Overloading operator *
c5=c1*c2;
c5.getData();

//Overloading operator /
c6=c2/c1;
c6.getData();
getch();
}
```

C) Construct a program to overload logical AND (&&) operator using friend function.

```
#include <iostream>
Using namespace std;
class MyClass {
private:
   bool value;

public:
   MyClass(bool val) : value(val) {}

   friend bool operator&&(const MyClass& obj1, const MyClass& obj2);
};

bool operator&&(const MyClass& obj1, const MyClass& obj2) {
   return obj1.value && obj2.value;
}

int main() {
   MyClass obj1(true);
   MyClass obj2(false);
   MyClass obj3(true);

   // Using the overloaded && operator
   if (obj1 && obj2) {
         cout << "obj1 && obj2: true" << endl;
   } else {
         cout << "obj1 && obj2: false" << endl;
   }
```

```
    if (obj1 && obj3) {
        cout << "obj1 && obj3: true" << endl;
    } else {
        cout << "obj1 && obj3: false" << endl;
    }

    return 0;
}
```

c) A friend function cannot be used to overload the assignment operator (=). Explain why?

In C++, the assignment operator (=) cannot be overloaded using a friend function. The assignment operator is a special operator that is used to assign the value of one object to another object of the same class. It is typically implemented as a member function of the class.

The assignment operator is implicitly called when you assign one object to another using the = operator. For example, obj1 = obj2; calls the assignment operator. Friend functions are not invoked implicitly in this manner. They require explicit function calls, which means that using a friend function to overload the assignment operator would break the typical syntax and behavior of object assignment.

D) When an operator is overloaded, does it lose any of its original functionality?

When an operator is overloaded, it is possible for it to lose some or all of its original functionality, depending on how the operator is defined in the overloaded implementation.
By overloading an operator, you provide a new definition for that operator when applied to specific types or classes. This allows you to customize the behavior of the operator to fit the requirements of your program or data types.

However, it's important to note that the original functionality of an operator is not inherently lost when it is overloaded. The original functionality of an operator is preserved for the types or classes for which it was originally defined. When the operator is used with those types or classes, the original behavior will still apply.

Muhammad Nazim Uddin
Lecturer(Adjunct)
Dept of CSE,IIUC